Molecular dynamics simulations in 2D

Tomasz Chwiej

8th December 2025

1 Introduction

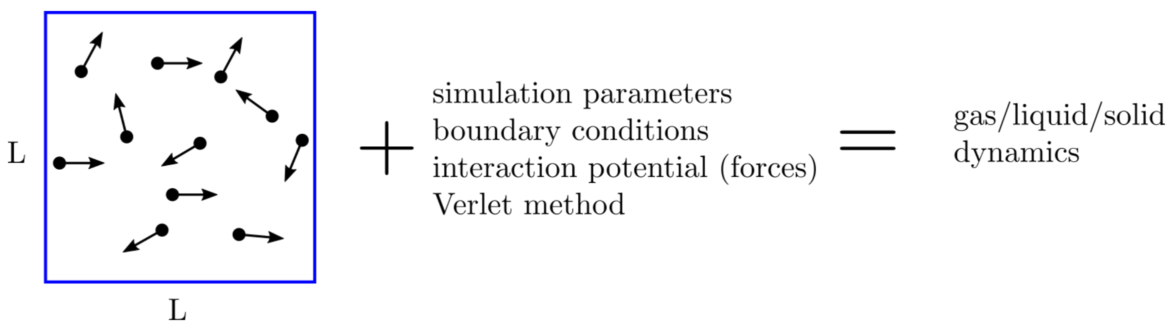

Figure 1: The dynamics of N particles confined in a square box of width L is simulated to find the properties of gas/liquid/solid. system.

In project we will simulate the dynamics of N argon atoms confined in square box of width L [Fig.1]. Interactions between atoms are described by Lennard-Jones model potential which gradient gives the forces exerted on atoms. These internal forces enter the equations of motion which in turn will be solved with velocity-Verlet method. In order to minimize the finite size effects, and to mimic the infinite system, the periodic boundary conditions are used. Although implementation of linked-cells-method would significantly reduce the time of computations, due to lack of time we wouldn't implement it, instead we use smaller number of particles (hundreds instead of thousands).

The temporary state of the system is defined by all positions and velocities of atoms. To change their positions we need to calculate also the forces, accelerations and half-time step velocities which must be stored until we proceed further in Verlet algorithm. For this reason it is advisible to construct the data structure for each atom separately, in C++ this could be a class containing all needed information

```
class PARTICLE{
        public:
                double mass;
                double x,y;
                double vx,vy;
                double vxp,vyp;
                double ax,ay;
                double fx,fy;
};
```

In the main program all the atoms' information can be comfortably kept in an array of elements being the instantiations of this class

```
vector<PARTICLE> particles(N);
```

This data structure is used in some pseudocodes presented below and is advised to use it in project.

1.1 Effective (reference) units and how to use them

In project we simulate dynamics of argon atoms, therefore it's natural to choose the effective units for the mass, the energy (potential) and the length adequate for argon atoms mixture with embedded Lennard-Jones interaction potential V_{LJ} (ϵ, σ are V_{LJ} parameters): $m_{ref} = m$, $E_{ref} = \epsilon$ and $r_{ref} = \sigma$ and then add to this list the others: reference temperature $T_{ref} = 119[K]$, time reference $t_{ref} = \sqrt{m_{ref} r_{ref}^2 / E_{ref}}$, Boltzmann constant $k_{ref} = k_B$

Practise:

The effective/reference units are introduced at the begining of simulation program and then are consequently used in calculations. Actually we need to rescale only one quantity: temperature which is expressed in Kelvins, the others can be defined right away in effective units:

$$T\,[T_{ref}] \leftarrow \frac{T[K]}{T_{ref}} \quad \text{(rescale temperature)} \tag{1}$$

$$m\,[m_{ref}] \leftarrow \frac{m}{m_{ref}} = 1 \tag{2}$$

$$k_B\,[k_{ref}] \leftarrow 1 \tag{3}$$

$$\epsilon \leftarrow 1 \tag{4}$$

Remark1:

Rescale temperature at the begining and use Eqs. 2, 3 and 4 every time these quantities occur in calculations (equations), other quantities are then expressed in effective units.

Remark 2:

If a value of physical quantity shall be expressed in SI units just multiply this quantity by corresponding effective unit also expressed in SI, for example

$$T\,[K] \leftarrow T\,[T_{ref}] \cdot T_{ref}, \qquad (T_{ref} = 119\,\mathrm{K}) \tag{5}$$

1.2 Initial positions and velocities

1.2.1 initial positions

Before MD simulation will be ready to start one must distribute the particles over the computational box and give them some velocities. Positions $\vec{r}_i = (x_i, y_i)$ can be set according to the output of random number generator of uniform distribution (in box $L \times L$), but this approach has embedded danger of placing two particles very close to each other what can make their interaction energy (LJ potential) very high and hence enforce to use extremely small time step. This issue can be bypassed by employing one of the adaptive time step methods but obviously this approach would require more time to implement it. Another way is much more simple as it needs to place the particles at the nodes of some arbitrary lattice (e.g. square, rectangular, triangular, etc.), then the spatial separation largely limits the interaction energy. Example is shown in Fig.2, we use this approach in project. This can be done in few steps

- define the distance bewteen nodes (the same in x/y direction)

$$\Delta = \frac{L}{ceil\left(\sqrt{N}\right)} \tag{6}$$

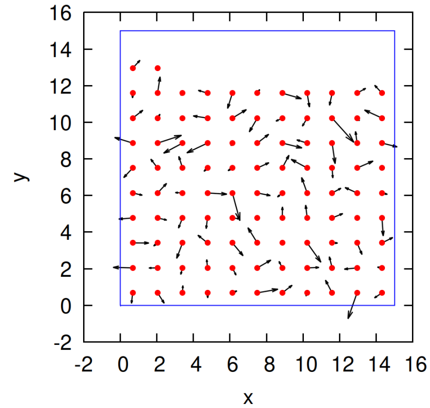

Figure 2: Example spatial distribution of particles which are placed at the nodes of square lattice, the arrows show directions and magnitudes of their velocities obtained from Maxwell-Boltzmann velocity distribution. Blue lines mark the edges of computational box $L \times L$.

where: L - width of box, ceil(arg) - the smallest integer equal or greater than the argument of function

- calculate the number of nodes in one dimension

$$k = floor\left(\frac{L}{\Delta}\right) \tag{7}$$

where: floor(arg) - the greatest integer equal or lower than the argument of function

- determine positions of each particle

for i=0 to N−1 do

$$particles[i].x \leftarrow \frac{\Delta}{2} + (i\%k) \cdot \Delta$$

$$particles[i].y \leftarrow \frac{\Delta}{2} + floor\left(\frac{i}{k}\right) \cdot \Delta$$

end do

whete % - is modulo division

1.2.2 initial velocities

Now we set particles' velocities. This step could be potentially skipped since attractive and repulsive interactions will force particles to move and further temperature tuning would appropriately adjust the velocities. Instead we may approach demanded temperature by setting the velocities according to Maxwell-Boltzmann (MB) distribution which in d-dimensions is defined as follows

$$f^{MB}_{|\vec{v}|}(v) = \left(\frac{m}{2\pi k_B T}\right)^{\frac{d}{2}} e^{-\frac{m\vec{v}^2}{2k_B T}} \tag{8}$$

where: m - mass of particles, k_B - Boltzmann constant, $\vec{v}$ - velocity vector, d - dimensionality of problem ($d = 2$ in our project). It describes the distribution of velocity's magnitude, but the directions of velocities are hidden. Here we have two choices: (i) draw random number from distribution $f^{MB}_{|\vec{v}|}$ (magnitude of velocity) and choose the direction randomly from uniform distribution, or, (ii) draw velocity's components v_x, v_y directly from two distributions $f^{MB}_{v_x}(v_x)$ and $f^{MB}_{v_y}(v_y)$. Let's use the latter approach, for this purpose we need MB distribution including velocity components

$$f_{\vec{v}}^{MB}(v_x, v_y) = \underbrace{\left(\frac{m}{2\pi k_B T}\right)^{\frac{1}{2}} e^{-\frac{mv_x^2}{2k_B T}}}_{f_{v_x}^{MB}} \cdot \underbrace{\left(\frac{m}{2\pi k_B T}\right)^{\frac{1}{2}} e^{-\frac{mv_y^2}{2k_B T}}}_{f_{v_y}^{MB}} = f_{v_x}^{MB}(v_x) \cdot f_{v_y}^{MB}(v_y) \tag{9}$$

$$\int_{-\infty}^{\infty} dv_x f_{v_x}^{MB}(v_x) \cdot \int_{-\infty}^{\infty} dv_y f_{v_y}^{MB}(v_y) = 1 \quad (\text{ normalization}) \tag{10}$$

We recognize that both $f_{v_x}^{MB}$ and $f_{v_y}^{MB}$ are representative of normal distribution: $\mathcal{N}(\mu, \sigma)$ with $\mu = 0$ and $\sigma = \sqrt{\frac{k_B T}{m}}$. **Remark: in effective units $m \leftarrow 1$, $k_B \leftarrow 1$ and $T \leftarrow T[K]/T_{ref}$.** Thus to get the components of velocity (v_x, v_y) drawn from Maxwell-Boltzmann distribution one needs to draw two numbers (s_x, s_y) from normal distribution $\mathcal{N}(0, 1)$ and scale them with σ: $v_x \leftarrow \sigma \cdot s_x$ and $v_y \leftarrow \sigma \cdot s_y$. Following pseudocode uses normal distribution generator (which transforms the uniformly distributed random numbers) provided by C++ to generate (s_x, s_y)

```
#include<random>

default_random_engine generator;                  //delivers uniform distribution U(0,1)
normal_distribution<double> normal(0.0,1.0);      //set parameters for N(0,1)
σ = sqrt(k_B T / m)
for i=0 to N-1 do

        s_x ← normal(generator) //draw 1-st random number
        s_y ← normal(generator) //draw 2-nd random number
        particles[i].vx ← σ · s_x
        particles[i].vy ← σ · s_y

end do
```

1.2.3 removing initial velocity drift

Although particles have been given initial velocites from proper distribution, we must keep in mind that for small number of particles the velocity of center-of-mass (v_{cm})

$$\vec{v}_{cm} = \frac{1}{N} \sum_{i=0}^{N-1} \vec{v}_i \tag{11}$$

will be non-zero giving artificial drift of a system which distorts any physical quantites depending on particles velocities (e.g. kinetic energy, temperature). To remove $\vec{v}_{cm}$ we must find its components and then subtract them from particles' velocities (this rescales a little the initial temperature, which must be then restored by some means):

```
v_cm,x = 0
v_cm,y = 0
for i=0 to N-1 do
        v_cm,x ← v_cm,x + particles[i].vx/N
        v_cm,y ← v_cm,y + particles[i].vy/N
end do
```

```
for i=0 to N-1 do
    particles[i].vx ← particles[i].vx − v_{cm,x}
    particles[i].vy ← particles[i].vy − v_{cm,y}
end do
```

1.3 Lennard-Jones potential, potential energy and forces

The dynamics of particles are mainly govern by interparticle interactions, these will be modelled by shifted Lennard-Jones potential (see lecture on molecular dunamics for more details), that is, each atom interacts only with these atoms for which their interparticle distance r_{ij} is smaller than arbitrary chosen r_{cut}

$$U_{LJ}^{shift}(r_{ij}) = U_{LJ}(r_{ij}) - U_{LJ}(r_{cut}) - \left.\frac{\partial U_{LJ}(r_{ij})}{\partial r_{ij}}\right|_{r_{ij}=r_{cut}} \cdot (r_{ij} - r_{cut}) \tag{12}$$

where $U_{LJ}(r_{ij})$ is the Lennard-Jones in its standard form

$$U_{LJ}(r_{ij}) = 4\epsilon\left(\frac{\sigma^{12}}{r_{ij}^{12}} - \frac{\sigma^{6}}{r_{ij}^{6}}\right) \tag{13}$$

Remark: in effecive units use $\epsilon \leftarrow 1$ and $\sigma \leftarrow 1$.

Because we use the periodic boundary conditions (minimal coupling image), two atoms from opposite ends of box may interact, consequently the effective distance bewteen them (r_{ij}) must be adequately scaled. In order to include this information in r_{ij} we define two parameters

$$\alpha = \begin{cases} -1, & x_j - x_i > \frac{L}{2} \\ +1, & x_j - x_i \leq -\frac{L}{2} \\ 0, & otherwise \end{cases} \tag{14}$$

and

$$\beta = \begin{cases} -1, & y_j - y_i > \frac{L}{2} \\ +1, & y_j - y_i \leq -\frac{L}{2} \\ 0, & otherwise \end{cases} \tag{15}$$

Then the real distance between two atoms reads

$$r_{i,j} = \sqrt{(x_j - x_i + \alpha L)^2 + (y_j - y_i + \beta L)^2} \tag{16}$$

and the interaction as well as the force are nonzero **if and only if** $r_{ij} < r_{cut}$. This condition decreases the number of time-consuming calculations for more distant atoms which contributions to the force (and dynamics) are negligible.

The explicit form of U_{LJ}^{shift} is following

$$U_{LJ}^{shift}(r_{ij}) = 4\epsilon\left(\frac{\sigma^{12}}{r_{ij}^{12}} - \frac{\sigma^{6}}{r_{ij}^{6}}\right) - 4\epsilon\left(\frac{\sigma^{12}}{r_{cut}^{12}} - \frac{\sigma^{6}}{r_{cut}^{6}}\right) - 4\epsilon\left(-\frac{12\sigma^{12}}{r_{cut}^{13}} + \frac{6\sigma^{6}}{r_{cut}^{7}}\right)(r_{ij} - r_{cut}) \tag{17}$$

and the force exerted on i-th particle by j-th particle can be calculated using the chain rule

$$\vec{F}_{ij} = -\nabla_{\vec{r}_i} U_{LJ}^{shift}(r_{ij}) = -\frac{\partial U_{LJ}^{shift}(r_{ij})}{\partial r_{ij}} \nabla_{\vec{r}_i} r_{ij}, \qquad \left(\nabla_{\vec{r}_i} = \left[\frac{\partial}{\partial x_i}, \frac{\partial}{\partial y_i}\right]\right) \tag{18}$$

$$\frac{\partial U_{LJ}^{shift}(r_{ij})}{\partial r_{ij}} = 4\epsilon\left(-\frac{12\sigma^{12}}{r_{ij}^{13}} + \frac{6\sigma^{6}}{r_{ij}^{7}}\right) - 4\epsilon\left(-\frac{12\sigma^{12}}{r_{cut}^{13}} + \frac{6\sigma^{6}}{r_{cut}^{7}}\right) \tag{19}$$

$$\nabla_{\vec{r}_i} r_{ij} = \left[-\frac{x_j - x_i + \alpha L}{r_{ij}}, -\frac{y_j - y_i + \beta L}{r_{ij}}\right] \tag{20}$$

The total potential energy is the sum of all pair interactions (double counting is avoided)

$$E_{pot} = \sum_{i=0}^{N-1} \sum_{j=i+1}^{N-1} U_{LJ}^{shift}(r_{ij}) \tag{21}$$

and the force exerted on i-th particle is the sum of all partial contributions

$$\vec{F}_i = \sum_{\substack{j=0 \\ j\neq i}}^{N-1} \vec{F}_{i,j} \tag{22}$$

Now we may write piece of code calculating all internal forces, moreover, by exploiting third Newton law $\vec{F}_{ji} = -\vec{F}_{ij}$ we are able to reduce simply the number of required operations by factor 2.

```
zeroing all forces:
for i=0 to N-1 do
        particles[i].fx ← 0
        particles[i].fy ← 0
end do

calculate new forces and potential energy (at no additional cost):
E_pot ← 0
for i=0 to N-1 do
        x_i ← particles[i].x
        y_i ← particles[i].y

        for j=i+1 to N-1 do //avoid double counting (j>i)
                x_j ← particles[j].x
                y_j ← particles[j].y
                α ← Eq.14
                β ← Eq.15
                r_ij ← Eq.16

                IF (r_ij < r_cut)THEN //skip unnecessary computations
                        U_LJ^shift ← Eq.17
                        F_ij ← Eqs.18, 19, 20

                        add up partial contributions:
                        particles[i].fx ← particles[i].fx + F_x,ij
                        particles[i].fy ← particles[i].fy + F_y,ij

                        exploit 3-rd Newton's law:
                        particles[j].fx ← particles[j].fx − F_x,ij
                        particles[j].fy ← particles[j].fy − F_y,ij

                        potential energy:
                        E_pot ← E_pot + U_LJ^shift
                END IF
        end do
```

```
end do

calculate accelerations:
for i=0 to N-1 do
        particles[i].ax ← particles[i].fx/particles[i].mass
        particles[i].ay ← particles[i].fy/particles[i].mass
end do
```

In above pseudocode we use the information stored solely in the array *particles*, hence it is advisable to place it in separate function which will be called with single instruction.

1.4 Kinetic energy and instantaneous temperature

Besides the potential energy we need also the kinetic one. The total kinetic energy is the sum of kinetic energies of atoms

$$E_{kin} = \sum_{i=0}^{N-1} \frac{m_i(v_{x,i}^2 + v_{y,i}^2)}{2} \tag{23}$$

Kinetic energy may then be used in calculation of instantaneous temperature $\mathcal{T}$ from equipartition theorem

$$\mathcal{T} = \frac{E_{kin}}{N k_B} \tag{24}$$

Remark: in Eqs.23 and 24 use effective units $k_B \leftarrow 1$ and $m \leftarrow 1$. Implementation of both expressions in program is straightforward.

1.5 Time iteration, Verlet method and thermalization

We know how to calculate the forces and accelerations which are needed for solving the equations of motion. Now we may construct the final part of our simulation program i.e. implement the main time loop which iterates over subsequent time instants every time calling the Verlet method combined with Nose-Hoover thermostat. The following pseudocode is an adapted version of that presented at the lecture

```
initialization: particles[], ξ=0, Q, T, Δt, N_it, k_max = 5

calculate forces and accelerations: Sec.1.3

for it=1 to N_it
        time = Δt · it

        make 1-st step in Verlet method
        for i=0 to N-1 do
                particles[i].vxp ← particles[i].vx + Δt/2 (particles[i].ax − ξ · particles[i].vx)
                particles[i].vyp ← particles[i].vy + Δt/2 (particles[i].ay − ξ · particles[i].vy)
        enddo

        make 2-nd step in Verlet method
        for i=0 to N-1 do
                x ← particles[i].x + particles[i].vxp · Δt
                y ← particles[i].y + particles[i].vyp · Δt
```

```
            implement periodic BC:
            if (x < 0) x ← x + L
            else if (x > L) x ← x − L

            if (y < 0) y ← y + L
            else if (y > L) y ← y − L

            particles[i].x ← x
            particles[i].y ← y
      enddo
      3−rd step, calculate accelerations and E_pot using new positions:
            use pseudocode from Sec.1.3

      initialize new velocities with half−step approximations:
      for i=0 to N−1 do
            particles[i].vx ← particles[i].vxp
            particles[i].vy ← particles[i].vyp
      end do

      find new ξ and new velocities using k_max Picard subiterations:
      for k=1 to k_max do
            calculate total kinetic energy E_kin and instantaneous temperature 𝒯:
            E_kin ← Eq.23
            𝒯 ← Eq.24
            update ξ:
            ξ ← ξ + Δt/Q (E_kin − T·(N+1))  //T−is the demanded temperature
            update velocities:
            for i=0 to N−1 do
                  particles[i].vx ← 1/(1+Δt/2 ξ) (particles[i].vxp + Δt/2 particles[i].ax)
                  particles[i].vy ← 1/(1+Δt/2 ξ) (particles[i].vyp + Δt/2 particles[i].ay)

            end do
      end do

      E_tot ← E_kin + E_pot
      send output: it, time, E_kin, E_pot, E_tot, 𝒯

end do
```

1.6 Velocity distribution

The Maxwellian velocity distribution for 2D gas reads

$$f_v(v) = \frac{m}{k_B T} \cdot v \cdot e^{-\frac{mv^2}{2k_B T}} \tag{25}$$

Notice that Maxwellian distribution (Eq.25) differs from the Maxwell-Boltzmann one (Eq.8), the Maxwell-Boltzmann distribution is defined in the full 2D velocity space while Maxwellian velocity distribution is obtained from MB by integrating over full solid angle and thus is defined only for non-zero velocities (information about the direction of velocity is definitely lost).

We simulate the dynamics of group of atoms, which can be treated as atomic gas, provided that the mean distance between atoms is at least 10 times larger than the spatial size of single atom and the temperature is quite high i.e. the kinetic energy dominates the potential energy. Then we may reconstruct the Maxwellian velocity distribution as a histogram. First, for given (demanded) temperature T the characteristic velocity v_c is calculated, it is needed for estimation of the maximal velocity v_{max} that limits the range in histogram

$$2\frac{k_B T}{2} = \frac{m v_c^2}{2} \qquad \Longrightarrow \qquad v_c = \sqrt{\frac{2k_B T}{m}} \qquad \Longrightarrow \qquad v_{max} = 4 \cdot v_c \tag{26}$$

Second, one must define the number of bins in histogram n_v, calculate the width of single bin

$$\Delta v = \frac{v_{max}}{n_v} \tag{27}$$

and create an array in which the data will be gathered in the course of simulation. Accumulation of data in array can be done by using the following pseudocode - in fact it is shortened version of main time iteration loop, hence it just shows how to modify the main program to calculate the histogram

```
initialize: T, v_max , n_v , Δv
                    vector<double> v_hist[n_v]

zeroing the elements of array (remove numerical trashes):
for i=0 to n_v do
        v_hist[i] ← 0
end do

for it=1 to N_it

        make single step with velocity−Verlet method

        gather data for velocity histogram from all atoms:
        for i=0 to N−1 do
                vx ← particle[i].vx
                vy ← particle[i].vy
                v ← sqrt(v_x^2 + v_y^2)
                k ← floor(v/Δv)
                if(k < n_v) v_hist[k] ← v_hist[k] + 1/(N_it·N·Δv)
        end do

end do
```

The division by N_{it} and N means that the histogram is averaged over all iterations and for all atoms while Δv in denominator ensures that the histogram is normalized

$$\sum_{i=0}^{n_v} v_{hist}[k] \cdot \Delta v \approx 1 \tag{28}$$

in the same way as the analytical function (Maxwellian distribution)

$$\int_0^\infty f_v(v)dv = 1 \tag{29}$$

and both can be directly compared.
Remark:
Actually, normalization constant for velocity histogram is slightly smaller than one because the data for atoms having velocities greater than v_{max} are not stored, this can be prevented by extending the v_{max} but this requires increasing n_v to keep fine velocity resolution (width of single bin), and this in turn will require longer simulation time to accumulate statistically significant data. These parameters are commonly determined empirically.

2 Practical part

Implement on computer the pseudocodes for the Verlet method (Sec.1.5) and calculations of the forces/potential energy (Sec.1.3). At the begining assume following parameters (some of them will be changed):

- width of square box: $L = 25\,[r_{ref}]$
- number of atoms: $N = 201$
- time step: $\Delta t = 0.002\,[t_{ref}]$,
- number of time steps: $N_{it} = 10^4$
- temperature: $T = 300[K]$ (divide it by $T_{ref} = 119\,K$ to get the effective unit)
- mass of atom: $m = 1[m_{ref}]$
- cutting radius for interaction potential: $r_{cut} = 2.7\,[r_{ref}]$

Tasks to do:

1. Check the correctness of initial conditions
   Initialize the positions of atoms (Sec.1.2), save them to file and then plot these positions in order to check if they has been distributed correctly. Initialize the velocities using the pseudocode presented in Sec.1.2.2, remove the velocity drift (Sec.1.2.3) and calculate the instantaneous temperature $\mathcal{T}$ (Sec.1.4). Compare both temperatures, do they differ much or not? Example initial positions are shown in Fig.3.

2. Check the conservation of total energy in Verlet method
   (a) Set $N_{it} = 10^4$ and $Q = 10^{20}$ which turns off thermostat. Conduct the time simulation, every 10 time steps save to file kinetic, potential and total energies. Plot them as function of time in one figure. If everything is ok, the total energy will be constant while the kinetic and the potential ones will fluctuate. This simulation relates to the microcanonical ensemble.
   (b) Set $N_{it} = 5 \cdot 10^4$ and $Q = 10^{20}$, conduct simulation but every 10 time steps write to file the position of $l = N/2$ atom (initially it is located near the center of the box). Plot its trajectory. Example results for (a) and (b) are shown in Fig.4.

3. Check the correctness of thermalization process
   Set $N_{it} = 10^4$ and conduct simulations for coupling parameter: (a) $Q = 1$, (b) $Q = 0.1$ and (c) $Q = 0.01$. Plot instantaneous energies and temperature for each case in one figure. For each Q calculate the mean temperature of gas (average all instantaneous values). Example results for (a), (b) and (c) are shown in Fig.5.

4. Velocity distribution
   Set $Q = 10$ and make simulations for: (a) $N_{it} = 10^3$, (b) $N_{it} = 10^4$ and (c) $N_{it} = 10^5$. In each case gather the data for the velocity histogram using the information given in Sec.1.6, then plot

both, the velocity histogram and an analytical velocity distribution, in one figure.
Example results for (a), (b) and (c) are shown in Fig.6.

5. Crystalization in atomic system
Assume the following parameters: $Q = 1$, $T_{max} = 300\,K$, $T_{min} = 5\,K$, $n_T = 500$, $K_{it} = \frac{N_{it}}{n_T}$, $\Delta T = (T_{max} - T_{min})/n_T$. The demanded temperature T of atomic system now decreases in simulation depending on iteration index it in following manner
$$T(it) = T_{max} - \Delta T \cdot floor\left(\frac{it}{K_{it}}\right) \tag{30}$$
Implement the dynamical change of temperature given in Eq.30 in Your program and then conduct simulations for: (a) $N_{it} = 10^4$ and (b) $N_{it} = 10^5$ iterations, i.e. for fast and slow cooling, respectively. When the temperature eventually drops to T_{min} the atoms will freeze at some positions forming either local and global atomic structures of minimal potential energy. For (a) and (b) plot instantaneous temperatures in one figure, make plots of atomic positions for both cases.
Example results are shown in Fig.7.

3 Example results

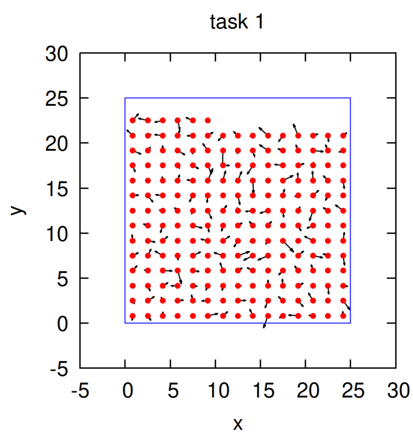

Figure 3: (Task 1) initial positions of atoms at nodes of regular grid. Arrows mark the velocity vectors.

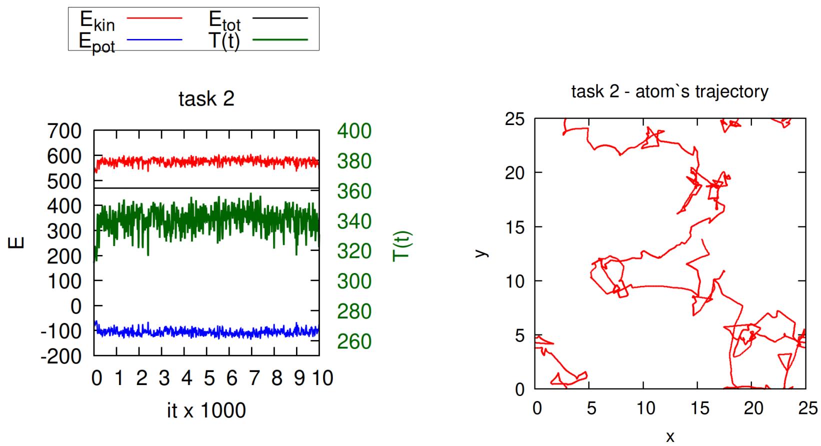

Figure 4: (Task 2) Energies and instantaneous temperature as functions of iteration for microcanonical ensemble (left) and the trajectory of arbitrary chosen atom (right). Notice that the trajectory has rather chaotic character, atom collides with others and rapidly changes the direction of motion, its motion has diffusive character. Due to quite low density of particles, the atom may pass through the box many times in short time.

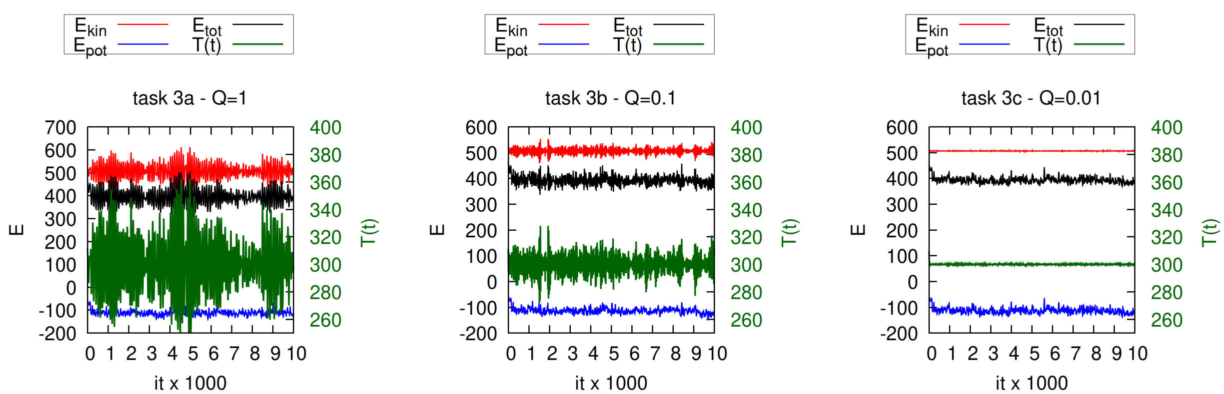

Figure 5: (Task 3) Energies and instantaneous temperature as functions of iteration for canonical ensemble and different coupling parameter Q (displayed on top of each subfigure). Notice that for the strongest coupling of system to heat bath ($Q = 0.01$) the temperature and kinetic energy are "almost" constant, or in other words, their fluctuations have very small amplitudes.

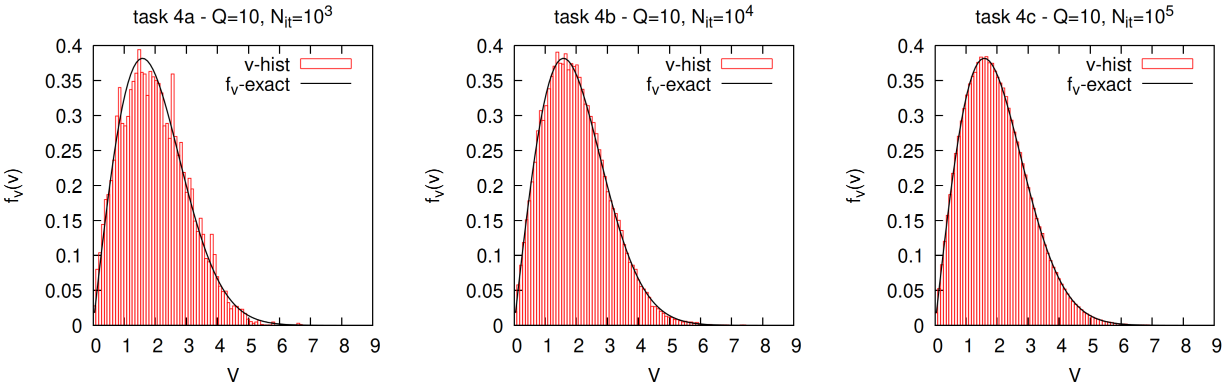

Figure 6: (Task 4) Velocity histograms for $Q = 10$ and $N_{it} = 10^3$, 10^4, 10^5. Even for large number of iterations there are some small discrepances between numerical and exact results, notice however that these could be the result of calculations made for not ideal gas system (interactions) as well. Nevertheless the following general rule applies: the longer time simulation proceeds the more data are gathered giving smaller fluctuations of calculated physical quantities.

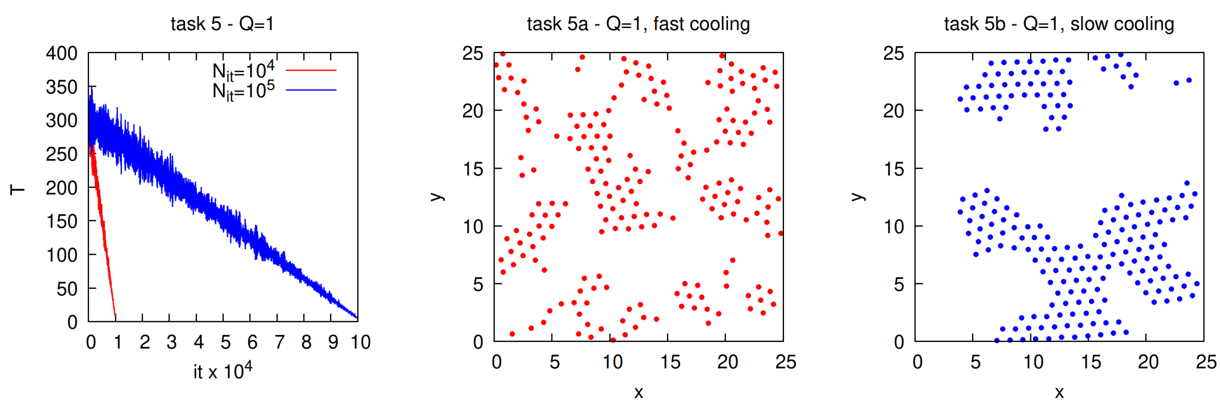

Figure 7: (Task 5) Fast cooling of atomic systems ($T_{max} = 300\,K \quad \rightarrow \quad T_{min} = 5\,K$) marked as red curve in left plot leads to freezing the atoms (middle plot) in many small nanoflakes whereas the slow cooling (blue curve in the left plot) gives the atoms more time to relax their potential energy leading eventually to formation of more regular atomic structure resembling the triangle lattice (right plot).